

1. 简答题

1.1. 软件系统架构

1. 【必考】如何进行质量属性方案建模？请使用"刺激-相应"图的格式进行建模 How to model quality attribute scenarios? Graphically model one quality attributes in "stimulus-response" format:

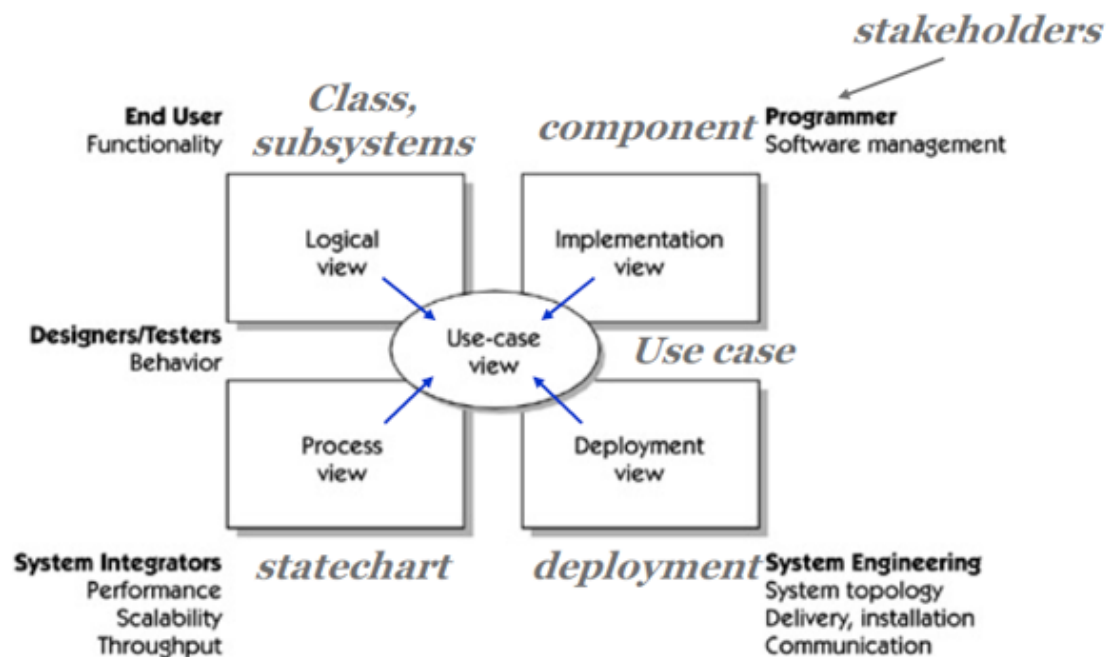
1. 【2015】availability and Performance
2. 【2017】 【2018】 availability and modifiability
3. 【2019】 interoperability and modifiability
4. 如何进行质量属性方案建模
 1. 刺激 (Stimulus)：到达系统时需要考虑的条件。
 2. 刺激源 (Source of Stimulus)：产生刺激的实体（人，系统或任何其他触发），可能是输入、信息等，对当前的状态的一个变化。
 3. 响应 (Response)：刺激到来后工件开展的行为。
 4. 响应度量 (Response Measure)：对刺激的响应以某种方法进行测量，以便可以测试需求（比如多长时间系统有反馈）
 5. 环境 (Environment)：发生刺激时系统的状况，例如系统正常运行、系统过载、系统受到攻击、系统网络出现故障等。
 6. 工件 (Artifact)：完成需求的整个系统或者系统的一部分（软件制品）。

质量属性	刺激源	刺激	工件	环境	响应	响应度量
Availability	Heartbeat 监视器	服务器无响应	处理器	正常操作	通知操作者继续操作	没有停机时间
Interoperability	车辆信息系统	发送当前位置	路况监控系统	系统运行前已知	路况监控结合当前位置和 Google 地图上的其他信息，并且进行广播	我们的信息在 99.9%的时间是正确的
Modifiability	开发者	希望修改 UI 界面	代码	设计时	进行修改和单元测试	在 3 个小时内完成
Performance	用户	发起事务	系统	正常操作	事务被处理	平均延迟不超过 2 秒
Security	来自偏远地区心怀不满的员工	尝试修改支付率	系统内数据	正常操作	系统存储修改追踪	正确数据在 1 天内被存储并且进行篡改身份识别
Testability	单元测试者	单元测试完成	单元测试代码	开发时	记录结果	3 小时内达到 85%的路径覆盖
Usability	用户	下载新应用	系统	运行时	用户高效地使用应用	在 2 分钟以内的实验

2. 【高频 2015 2017 2019】为什么软件系统架构需要使用不同视图来文档化？给出4种示例视图的名称和目的。 Why should a software architecture be documented using different differenet views? Give the name and purposes of 4 example views.

1. 原因
 1. 不同视图支持不同的目标和用户，突出不同的系统元素和关系
 2. 不同视图将不同质量属性暴露出不同的程度
2. 4种视图(了解):

1. 模块视图 Module View: 提供一组连贯职责的实现单元
 2. 组件和连接器视图 C & C View: 显示具有某些运行时存在的元素
 3. 分配视图 Allocation View: 描述了软件单元到软件开发或执行环境元素的映射
 4. 质量视图 Quality Views, 安全视图、性能视图、可靠性视图、沟通视图、异常（错误处理）视图
 5. 组合视图: 将上述视图进行组合
3. 【2017】 【2019】描述 4+1 视图 Describe 4+1 view(掌握绘图): 答案如上
1. 逻辑视图: 描述了对架构而言重要的元素和他们之间的关系（功能需求）
 2. 过程视图: 描述了元素之间的并发和交互。
 3. 物理视图: 描述了主要过程和组件是如何被映射到硬件上的。
 4. 发展视图: 描述了软件组件的内部组织联系（比如使用配置管理工具存储）。
 5. 用例场景（Use Case）: 捕获架构需求，与一个或多个特定视图相关。



4. 【高频 2015 2017 2018】什么区分了软件产品线架构和单个软件产品架构? What distinguishes an architecture for a software product line from an architecture for a single product?
1. 产品线的目的: 实现高可重用性、高修改性。
 2. 产品线之所以有效是因为通过重用可以充分利用产品的共性, 从而产生规模的经济性。
 3. 在产品线架构中有一组明确允许发生的变化, 然而对于常规架构来说, 只要满足了单个系统的行为和质量目标, 几乎任何实例都是可以的。因此, 识别允许的变化是架构责任的一部分, 同时还需要提供内建的机制来实现它们。
5. 【高频 2015 2017】将以下每个问题（左侧）与解决该问题的架构风格/视图（右侧）对应起来。列出每个样式类别的四个视图。Map each of the following questions (on the left) with the architectural style/view (on the right) that addresses the question. List four views of each category of style.
1. 连线题
 1. 它是如何构建为一组实现单元的? How it is structured as a set of implementation of units(Module Styles)
 2. 它是如何构建为一组具有运行时行为和交互的元素的? How it is structured as a set of elements that have runtime behavior and interactions?(Component-Connector Styles)
 3. 它与环境中的非软件结构有何关系? How it relates to non-software structures in its environment?(Allocation Styles)

2. Module Styles: 分解视图、使用视图、泛化视图、分层视图、领域视图、数据模型视图
 3. Component-Connector Styles: 管道-过滤器视图、客户端-服务器视图、点对点视图、面向服务视图、发布-订阅视图
 4. Allocation Styles: 部署视图、安装视图、工作分配视图、其他分配视图。
6. 【2015】【2017】简要描述软件架构过程中的一般活动，以及每个活动的主要输入和输出。
Briefly describe the general activities in a software architecture process, and the major inputs and outputs at each activity.

1. 长答案

1. 创建商业案例 输入：问题域，输出：商业案例
2. 了解用户需求 输入：用户的模糊需求 输出：架构攸关的需求
3. 创建或选择体系结构 输入：策略、模式和备选场景 输出：被选中的策略、模式和场景
4. 沟通体系结构 输入：架构设计文档 输出：
5. 分析和评估体系结构 输入：架构设计文档 输出：
6. 实现体系结构 输入：架构设计文档 输出：架构设计的具体实现
7. 保证体系结构的一致性 输入：架构设计的具体实现 输出：保持一致的架构设计具体实现

2. 短答案(背)

1. 识别ASRs

1. 输入：无
2. 输出：优化的质量属性场景

2. 架构设计

1. 输入：优化的质量属性场景、需求和约束、模式和决策
2. 输出：一组候选视图的草图（模式决定）

3. 架构文档化

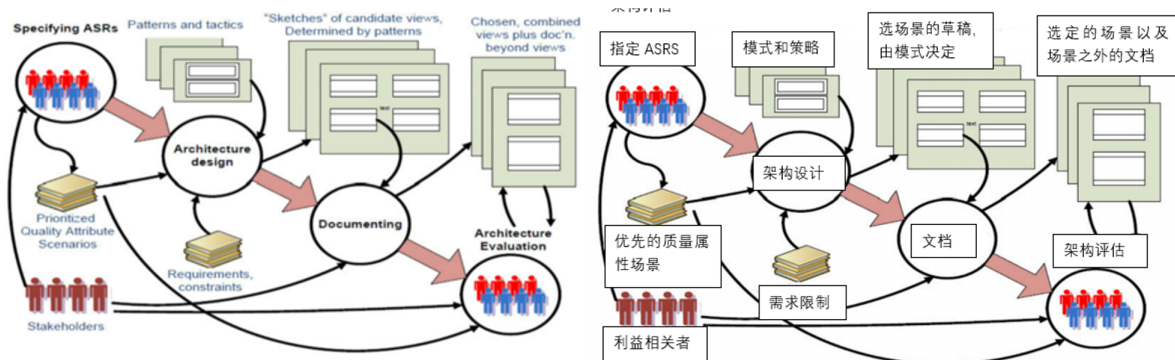
1. 输入：一组模式决定的草图（由模式决定）
2. 输出：View & Beyond

4. 架构评估

1. 输入：View & Beyond、优化的质量属性场景
2. 输出：View & Beyond

3. 其他

1. 通过StackHolder获取到ASRs（架构攸关需求）
2. 通过分析得到Prioritized Quality Attribute Scenarios(高优先级质量属性解决方案)和 Requirements, Constraints(需求和约束)
3. 将上述部分，结合模式和策略，综合可以得到架构的设计
4. 根据架构的设计得到由模式决定的候选视图的示意图，之后完成文档化
5. 选择、组合视图，将文档进行进一步的评估，这一部分需要StackHolder的参与、也需要 Prioritized Quality Attribute Scenarios和文档等作为参考。



7. 【2015】【2017】描述架构权衡分析方法 (ATAM) 过程的每个阶段生成的输出。Describe the outputs generated from each phase of Architecture Tradeoff Analysis Method(ATAM) process.

1. 阶段-0: 准备和建立团队(输入是架构设计文档)
 1. 评估计划
2. 阶段-1: 评估-1
 1. 架构的简明介绍
 2. 业务目标（驱动因素）的阐释
 3. 作为场景实现的特定质量属性要求的优先级列表
 4. Utility Tree 效用树
 5. 风险和无风险点
 6. 敏感和权衡点
3. 阶段-2: 评估-2
 1. 涉众们的优先级场景列表
 2. 风险主题和业务驱动因素各自受到的威胁
4. 阶段-3: 后续
 1. 最终的评估报告
5. ATAM的整体输出
 1. 架构的简明介绍
 2. 业务目标（驱动因素）的阐释
 3. 表现为质量属性场景的优先质量属性要求
 4. 质量属性效用树(Utility Tree)
 5. 风险和非风险组
 6. 风险主题组(共同的潜在问题或系统性缺陷将风险分组为风险主题)
 7. 将架构映射到质量属性
 8. 敏感点和权衡点
 9. 最终的评估报告

8. 【2019】描述在ATAM的每一个过程中 有哪些Stack holder和他们的职责

1. 阶段-0: 准备和建立团队
 1. 参与者: 评估团队领导和关键项目决策者
 2. 职责: 根据架构设计文档生成评估计划, 包括谁参加评估、如何何时何地开展评估、最后评估报告会被呈递给谁。
2. 阶段-1: 评估-1
 1. 参与者: 评估团队和项目决策者
 2. 职责:
 1. 第一步, 评估负责人介绍ATAM方法
 2. 第二步, 项目经理或客户从业务角度介绍业务驱动因素
 3. 第三步, 首席架构师介绍体系结构
 4. 第四步, 评估团队确定架构方法
 5. 第五步, 评估团队和项目决策者生成质量属性效用树(Utility Tree)
 6. 第六步, 评估团队分析架构方法
3. 阶段-2: 评估-2
 1. 参与者: 评估团队、项目决策者和项目涉众
 2. 职责:
 1. 第一步, 评估负责人介绍ATAM方法和之前已经取得的成果
 2. 第七步, 涉众头脑风暴并确定场景优先级
 3. 第八步, 评估团队分析架构方法, 类似第六步

4. 第九步，评估团队展示评估结果，并呈递给涉众

4. 阶段-3：后续

1. 参与者：评估团队、主要涉众

2. 职责：评估团队制作最终评估报告，发给主要涉众审核通过后，将报告呈递给委托评估的人。

9. 【2015】 【2019】 软件架构来自哪里？列举五种可能的软件架构的来源 Where do software architecture come from? List five possible sources of software architecture.

1. NFRs

2. ASRs

3. 质量需求

4. 涉众，组织

5. 技术环境

6. 业务目标

7. 商业与技术决策组合

10. 【2015】 【2019】 解释代理架构模式的上下文、好处和局限性。 Explain the context, benefits and limitations of Broker Architecture Pattern.

1. 上下文：多个同步或异步交互的远程对象组成的系统，broker模式已定义了运行时组件broker，它协调多个客户机和服务器之间的通讯。

2. 好处：提高了Client和Server之间的交互性、提高可伸缩性和可扩展性、解决了单体应用的性能瓶颈、大规模集群的性能提高，但是单点性能会下降。

3. 局限性：代理增加了前期复杂度、可能成为通信的屏障、可能成为攻击的目标、难以测试。

4. SOA延续了broker的思想，查找服务和使用服务都要通过broker，而SOA只在查找式通过register，分散了broker的职责，降低了单点风险。

11. 【2018】 Layered pattern 和 Multi-tier pattern 的区别

1. Layered Pattern是Module Style，而Multi-tier Pattern是Allocation Style

2. Layered Pattern是将任务拆解成一个个处于特定抽象级别的子层次，每层为下一层提供更高层次的服务，核心是关注点分离。

3. Multi-tier Pattern中的层是逻辑的组合，没有层次模式的强依赖关系，在不同部署环境中分层不同但是软件完成的内容一致。

12. 【2017】 【2019】 在设计软件时应用了哪些通用设计策略？为每个策略提供一个带有软件架构的简明工作示例。 What are generic design strategies applied in designing software? Give a concise working example with software architecture for each strategy.

1. 抽象：使用抽象让设计师关注本身结构而不关心实现，比如将系统抽象为组件和连接件或抽象为模块。

2. 分解：针对某一个系统关注点分解后处理，比如将整个系统分解或将某个模块分解。

3. 分而治之：将每个模块分别处理

4. 生成与测试：将一个特定的设计看作是一个假设；根据测试路径生成测试用例。

5. 迭代与细化：使用迭代的方法，ADD方法多次迭代直到满足所有ASR

6. 复用元素：重用在设计过程中出现了可以复用的元素，重用现有架构

13. 【2017】 【2019】 什么是ASR？列出提取和识别ASR的四种来源和方法。 What are ASR? List four sources and methods for extracting and identifying ASRs.

1. ASRs 架构攸关需求是对体系结构产生深远影响的需求的需求

2. 四种来源和方法：

1. 从需求文档中收集ASR：MoScoW方法和用户故事

2. 通过采访涉众来收集ASR：质量属性工作坊(QAW)

3. 通过了解业务目标来收集ASR：

4. 通过质量属性实体树(Utility Tree)来管理ASR：通过方案量化描述需求后，逐渐对质量属性进行分解细化，直到包含量化指标为止。

14. 【2017】 【2019】典型的软件架构文档包中应该包含哪些内容？简要描述每个组件及其用途。
What should be included in a typical software architecture documentation package? Briefly describe each component and its purpose.
1. 包含View和Beyond
 2. Beyond部分：
 1. 文档路线图：包含了范围和总结、简单摘要等。
 2. 视图的文档组织方式：描述了本文档中视图是如何组织的。
 3. 系统概述：从整体上描述了当前架构的简要说明、业务目标(驱动因素)等等。
 4. 视图之间的映射关系：描述了不同视图之间的映射关系。
 5. 系统原理：从整体上描述了当前架构的设计原理。
 6. 目录-索引、词汇表、首字母缩略词表。
 3. View部分
 1. styles and views(体系结构风格和视图)
 2. Structural views
 1. module viwes
 2. C & C views
 3. Allocation views
 3. Quality Views
 4. 每一个View的内容
 1. 主要介绍：显示视图的元素和关系，以及图例
 2. 元素介绍，详细介绍第一部分中描述的元素、元素属性、关系属性和元素接口和行为。
 3. 上下文图：描述系统如何与环境相关
 4. 可变性指南：告知视图中可能出现的变化
 5. 基本原理：解释设计如何映射在视图中，以及其合理性。
15. 【2015】描述架构设计中架构模式和决策之间的关系。给出可以在架构设计过程中使用的任何四种决策的名称，并描述每种决策的目的。Describe the relationships between architectual patterns and tactics in architecture design. Give name of any four tactics that can be used during architecture design and describe the purpose of each of them.
1. 架构模式与决策之间的关系
 1. 决策比样式更简单，仅有单一的结构或机制来应对单一的架构驱动
 2. 决策是构成架构模式的重要组成部分
 3. 架构模式通常将许多个决策组合在一起。
 4. 大多数架构模式都包含不同的决策，这些决策可能有共同的目的或者常被用于实现不同的质量属性。
 5. 决策和架构模式共同构成了软件设计时的工具。
 2. 四种决策的名称
16. 【2015】简要描述面向服务架构 (SOA) 的基本原则，并讨论 SOA 对互操作性、可伸缩性和安全性等质量属性的影响 Briefly describe the fundamental principles of Service Oriented Architecture(SOA) and discuss the impact of SOA on quality attributes like interoperability, scalability and security
1. SOA的基本原则
 1. 服务契约:服务按照描述文档所定义的服务契约行事
 2. 服务封装:除了服务契约所描述内容，服务将对外部隐藏实现逻辑
 3. 服务重用:将逻辑分布在不同的服务中，以提高服务的重用性
 4. 服务组合:一组服务可以协调工作，组合起来形成定制组合业务需求
 5. 服务自治:服务对所封装的逻辑具有控制权
 6. 服务无状态:服务将一个活动所需保存的资讯最小化
 2. SOA对互操作性的影响

1. SOA具有更高的互操作性：符合开放标准，可以更好的重用服务
2. 支持服务的自动识别、发现、注册和调用等等
3. SOA对可伸缩性的影响
 1. SOA具有更高的可伸缩性：服务自身高内聚、服务间松耦合，最小化维护的影响
 2. 但是SOA也会带来系统复杂度较高的问题
4. SOA对安全性的影响：
 1. 中间件可能会成为性能的瓶颈
 2. ESB等中间件都可以成为被攻击的目标
 3. 多服务导致攻击的跟踪、溯源和防御成为困难。
17. 【2019】微服务和SOA的区别，相同点
 1. 相同点：微服务和SOA都是分布式架构，微服务是SOA的一种扩展，都包含了服务契约、服务封装、服务重用、服务组合、服务自治和服务无状态等基本特点。
 2. 微服务去掉了SOA架构中的ESB，采用轻量级通信机制(HTTP、REST)进行服务之间的通信。
 3. 微服务的管理和部署结合DevOps实现自动化，可以对服务进行自动化部署和监控预警。
 4. 微服务引入了API网关、对客户端屏蔽访问各项服务的问题。
 5. 微服务引入了熔断器：避免出现服务失效或网络问题等导致的级联故障。
18. 【2017】软件设计的三个变化维度，每个维度的变化点。不同的绑定时间如何影响可修改性和可测试性。
 1. 三个变化维度：
 1. 面向对象 OOP，强调重用性、灵活性和扩展性。
 2. 面向方面 AOP，满足扩展的需求，可以在程序中自由的扩展功能
 3. 面向服务 SOA，是系统发布功能的一种方式，且基于这种方式下不同的系统之间可以有有效的沟通、协作。
 2. 设计时，开发时，测试时，发布时，运行时：可修改性降低，可测试性升高
19. 【2018】软件架构的关注点有哪些？利益相关方有哪些？
 1. 软件架构的关注点
 1. 利益相关者 Stakeholders addressed
 2. 解决的问题 Concerns addressed
 3. 语言，建模技巧 Language, modeling techniques
 4. 决策，模式 Tactics, Pattern
 2. 利益相关方有哪些？
 1. 顾客 Customer
 2. 用户 User
 3. 建筑师 Architect
 4. 需求工程师 Requirements engineer
 5. 设计师 Designer
 6. 实施者 Implementer
 7. 测试师，集成师 Tester, integrator
 8. 维护者 Maintainer
 9. 产品经理 Product manager
 10. 质量保证人 Quality assurance people
20. 【2018】Software requirements, Quality attributes, ASRs 的区别和联系
 1. 软件需求包括功能性需求和非功能性需求（又称质量需求）
 2. 质量属性是由软件的业务目标所决定，在功能性需求的基础上提供的整个系统的合乎需求的特性，是非功能需求的一种反应。
 3. ASRs架构攸关需求是对于体系结构有着深远影响的需求，肯定是软件需求的一部分。
21. 【2018】描述ADD过程
 1. 确定有足够的需求信息

2. 选择要分解的系统要素
 3. 确定所选的元素的ASR
 4. 选择符合ASR的设计
 1. 找出设计问题
 2. 列出子关注点替代模式/决策
 3. 从清单中选择模式/决策
 4. 确定模式/决策与ASR之间的关系
 5. 记录初步的架构视图
 6. 评估并解决不一致的问题
 5. 实例化架构元素并分配职责
 6. 实例化元素定义接口
 7. 验证和完善需求
 8. 重复进行2-7步直到满足所有的ASR
22. 【2019】 Architecture, structure和Design的区别?
1. Design 包含 Architecture, Architecture 包含 Structure
 2. 结构是静态的、逻辑的, 是关于系统如何构成的
 3. 体系结构除包含架构, 还会包含组件之间的相关的关系结构, 并定义一些动态的行为。
 4. 体系结构是关于软件设计的, 所有体系结构都是设计, 但是不是所有的设计都是体系结构, 体系结构是软件设计的一个部分
23. 【2018】【2019】 Risks, Sensitivity Points, Trade-Off Points分别是什么? 各举一个例子。
1. 识别风险: 发现可能对所需质量属性产生负面影响的架构决策, 例如使用分层模式可能带来性能损耗。
 2. 发现权衡: 影响多个质量属性的架构决策, 例如使用分层模式可能会带来性能损耗, 但是也会解耦增加系统的可修改性。
 3. 发现敏感点: 特定质量属性对其敏感的架构决策, 比如在对性能敏感的系统中, 决定使用缓存中间件。
24. 软件架构风格
1. module Styles: 认为体系结构是由模块组成。模块是实现单元的集合, 它提供了一组一致的职责。
 2. C & C Styles 认为体系结构是由组件(主要的处理单元和数据存储)、连接件(组件之间的交互路径)组成的。
 3. Allocation Styles 认为体系结构是由软件元素(软件元素具有环境所需的属性)和环境元素(环境元素有提供给软件的属性)组成的, 展示了软件如何与环境关联。

1.2. 设计模式

1. 【2017】请至少说出三个面向对象的原则, 并解释它们如何应用于策略模式? Please name at least three Object-Oriented principles, and explain how they are applied in Strategy pattern?
 1. 单一职责原则
 2. 开闭原则
 3. 里氏代换原则
 4. 合成复用原则
2. 【2019】设计模式是什么? 举例说明类模式和对象模式的区别?
 1. 什么是设计模式:
 1. 设计模式是对被用来在特定场景下解决一般设计问题的类和互相通信的对象的描述, 包括模式名称、问题、解决方案和效果四部分。
 2. (PPT)设计模式(Design Pattern)是一套被反复使用、多数人知晓的、经过分类编目的、代码设计经验的总结, 使用设计模式是为了可重用代码、让代码更容易被他人理解、保

证代码可靠性。

2. 设计模式分类

1. 根据其目的(模式是用来做什么的)可分为创建型(Creational)、结构型(Structural)和行为型(Behavioral)三种:
 1. 创建型模式主要用于创建对象。
 2. 结构型模式主要用于处理类或对象的组合。
 3. 行为型模式主要用于描述对类或对象怎样交互和怎样分配职责。
2. 根据范围,即模式主要是用于处理类之间关系还是处理对象之间的关系,可分为类模式和对象模式两种:
 1. 类模式处理类和子类之间的关系,这些关系通过继承建立,在编译时刻就被确定下来,是属于静态的。
 2. 对象模式处理对象间的关系,这些关系在运行时刻变化,更具动态性。
3. 【2019】防御式编程是什么?断言和错误处理的区别?
 1. 可以预见到(至少预先推测到)问题所在,断定代码中每个阶段可能出现的错误,并作出相应的防范措施,来防止类似的意外的发生。
 2. 断言和错误处理的区别
 1. 断言是在开发期间使用的、让程序在运行时进行自检的代码,是对开发人员的警告,通常是一个子程序或宏。断言不可以有副作用。
 2. 错误处理是对预先已经考虑到的错误(如用户错误、程序错误、意外情况等等)按照流程进行处理。
4. 【2019】设计模式对MVC的影响?
 1. MVC模式使用了运行时、动态和相互之间的关系集成到开发框架中,是分层模式的变种。
 1. 分为model(业务逻辑)、view(处理用户展示,接收用户操作)、controller(对用户操作进行处理,将信息通知给model)(强调模块间约束关系,model不可以直接返回到controller)
 2. 优点:耦合性低,重用性高,生命周期成本低,部署快,可维护性高,方便管理
 3. 缺点:没有明确定义,不适于中小型应用程序,增加实现复杂度,视图和控制器过于紧密,视图对模型访问低效。
 2. 四人书中,MVC模式是观察者模式、策略模式和组合模式的演化,可能涉及到工厂模式和装饰器模式
 1. 基于推送-订阅模式
 2. 观察者: model发生变化通知controller,然后更新view
 3. 策略模式: controllers帮助views对不同用户的输入做不同的响应。
 4. 组合模式: 一组views
5. 【2019】策略模式和状态模式的区别?
 1. 在状态模式中,具体状态类的方法参数中包含上下文对象,需要在状态处理完成后完成状态切换。
 2. 在策略模式中,直接对上下文类调用set方法设置策略即可,不涉及到策略的切换。
6. 【2019】最小知识原则在设计模式中的应用?
 1. 中介者模式
 2. 外观模式

2. 设计题

1. 【2019】一个游戏,有几种人类角色:骑士、骑兵、步兵,持有不同的武器(矛、剑、斧),拥有fight方法;有一个非人类角色巨魔,可以持有武器,但攻击方法不同(beat)。现在希望让巨魔和其他人类角色一起进行游戏,并且要求有角色死亡时其他活着的角色要收到通知。运用设计模式进行设计,并画出类图。

2. 【2019】表驱动，参考PPT即可。
3. 【2019】架构设计，没太看懂.....大概是分析程序结构生成报告？
4. 【2018】设计一个飞行模拟软件，要求能模拟多种飞机的特性。为了在将来支持更多飞机种类，要求使用策略模式。画出架构图和类图
5. 【2019】一个买票系统的设计题，不同的角色有不同的打折方案，用策略模式设计，最后画图，还要说明策略模式的使用场景。

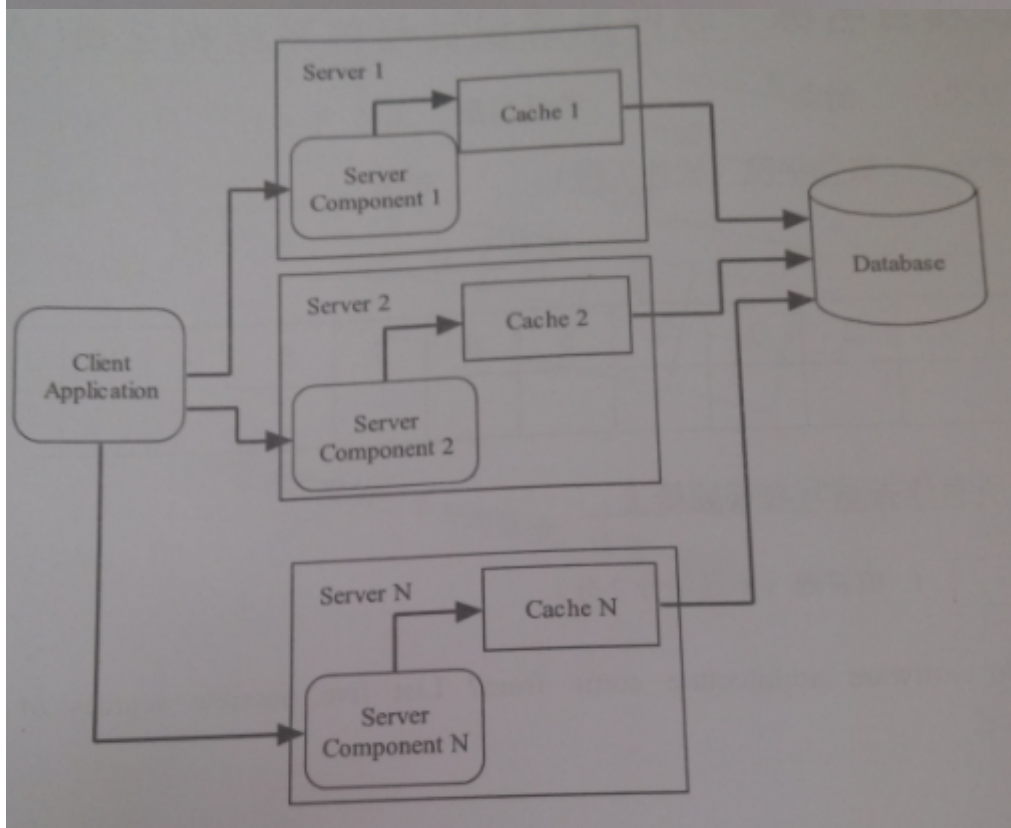
【2015】 【2019】

Distributed Cache Updates. Figure 1-1 below shows a distributed application with caches to improve its runtime performance. An online customer service system provides its clients access to their Bill Statement Information (BSI). The database stores BSI for clients. During a session, a client may require the BSI several times. In order to improve the performance, the BSI information for a particular client is cached. If the cache is loaded, then the BSI can be directly returned back to the client without access to the database. There are N servers clustered to serve the requests from clients. The server is load-balanced, and each time one server is scheduled for one client request. All the caches on N servers need to synchronize their states. If the client request changes the states of one cache, there are two requirements:

1. The changes are committed to the database;
2. All other cache states must be invalidated and they should reload the states from the database.

The current design in Figure 1 cannot meet the second requirement. Some important components are missing. Consider to apply architectural pattern(s) and tactic(s) to the design that can meet the second requirement, and complete the following questions:

1. (5 分) Identify extra component(s) that is (are) necessary for this design to meet second requirement. List each component name and its major functionalities.
2. (5 分) Draw sequence diagram to illustrate the communication between your devised components and original components in Figure 1.
3. (10 分) Assume that the servers are heterogeneous, and the protocols to access caches on different servers vary. For example, Server 1 supports Java RMI invocations to cache 1, Server 2 supports http JSP requests to cache 2, and server 3 supports SOAP requests to cache 3. It is further assumed that the cache interfaces are not consistent to each other due to the evolution of system development and legacy problems. Explain how the concept of connectors or web services can be applied to provide a generic invocation to caches? You can choose either connector or web services for discussion.



3. 其他知识点

1. 什么是软件系统架构：是结构和系统结构，包含了软件元素，这些组件的外部可视化属性和他们之间的关系（包含组件的行为）
2. 几个概念
 1. 功能性需求：定义系统必须做什么，并且强调系统如何提供价值给涉众。
 2. 质量需求：系统应在功能性需求之上提供的整个系统的合乎需求的特性。
 3. 策略：是影响质量属性相应控制的设计决策，比如冗余。
3. 架构模式
 1. 背景、上下文（Context）：世界上经常发生问题的场景。
 2. 问题（Problem）：在给定上下文中出现经过适当概括的问题。
 3. 解决方案（Element + Relations + Constraints）：针对问题的成功的经过适当抽象的解决方案。
4. ADD的输出
 1. 软件元素：完成各种决策和职责、定义属性并与其他软件元素相关以组成系统架构的计算或开发工件。
 2. 角色：一组相关的职责。
 3. 职责：软件提供的功能、数据或信息。
 4. 属性：有关软件元素的附加信息。
 5. 关系：两个软件元素之间如何相互关联和交互的定义。
5. 如何选择视图
 1. 构建涉众/视图表
 2. 合并视图
 3. 确定优先级和完成阶段
6. 什么是微服务：是分布式架构，SOA的一种扩展
 1. 微服务架构风格是一种将一个单一应用开发为一组小型服务的方法，每个服务运行在自己的进程中，服务间通信采用轻量级通信机制，这些服务围绕业务能力构建并可以通过自动部署机制独立部署。
 2. 微服务特点
 1. 服务颗粒化:服务粒度由业务功能决定，服务间尽可能解耦
 2. 责任单一化:单一职责原则，服务内尽可能内聚
 3. 运行隔离化:服务运行在各自进程中，互不影响
 4. 管理自动化:对服务提供自动化部署与监控预警能力，高效管理
 3. 微服务核心模式：针对采用微服务系统在特定问题所使用的程序的架构解决方案的集合
 1. 服务注册与发现
 2. API网关
 3. 熔断器
 4. 微服务的挑战
 1. 运维要求高:微服务数量多，部署与监控要求高
 2. 发布复杂度:部署环境多样化，网络性能系统容错、分布式事务等挑战
 3. 部署依赖强:服务间相互调用关系复杂，存在部署顺序依赖
 4. 通信成本高:跨进程调用比进程内调用消耗更多的资源
7. 面向对象设计原则

设计原则名称	设计原则简介	设计原则之间关系
单一职责原则 SRP Single Responsibility Principle	一个对象应该只包含 单一的职责 ，并且该职责被完整地封装在一个类中。	
开闭原则 OCP Open-Closed Principle	一个软件实体应该 对扩展开放，对修改关闭 。软件实体可以是一个软件模块、一个由多个类组成的局部结构或一个单独的类。	开闭原则是对单一职责原则的加强。
里氏代换原则 LSP Liskov Substitution Principle	所有引用基类（父类）的地方必须能 透明地 使用其子类的对象。 通俗表达 ：软件中如果能够使用基类对象，那么一定能够使用其子类对象。	里氏代换原则是开闭原则的具体实现。
依赖倒转原则 DIP Dependency Inversion Principle	高层模块 不应该依赖于低层模块 ，他们都应该依赖抽象。抽象不应该依赖于细节，细节应该依赖于抽象。 另一种表述 ：要针对接口编程，而不要针对实现编程。	依赖倒转原则是开闭原则的具体实现。
接口隔离原则 ISP Interface Segregation Principle	客户 不应该依赖 那些它不需要的接口。	拆分时需要满足单一职责原则
合成复用原则 CRP Composite Reuse Principle	在系统中应该尽量多实用组合聚合关联关系，尽量少甚至不使用继承关系。	
迪米特法则 LoD Law of Demter	在一个软件实体对其他实体的引用越少越好，或者说如果两个类不必彼此直接通信，那么这两个类就不应当发生直接的相互作用，而是通过引入一个第三者发生简介交互。	

8. 设计原则

1. 目标：开闭原则
2. 指导：最小知识原则
3. 基础：单一职责原则、可变性封装原则
4. 实现：依赖倒转原则、合成复用原则、里氏代换原则、接口隔离原则

9. 模式一般都有的缺点

1. 增加设计的复杂性和增加类的个数(增加辅助类)
2. 增加隔阂、方法调用，降低软件运行的效率，但是已经不是目前主要的问题

10. 引入设计模式的作用

1. 设计模式提供了与其他开发人员共享的词汇表。
2. 通过在模式级别（而不是实质性对象级别）进行思考，提高对体系结构的思考（但不要迷失在细节中）

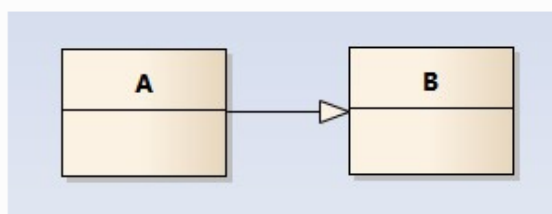
泛化关系(generalization)

类的继承结构表现在UML中为：泛化(generalize)与实现(realize)：

继承关系为 is-a 的关系；两个对象之间如果可以用 is-a 来表示，就是继承关系：（..是..）

eg：自行车是车、猫是动物

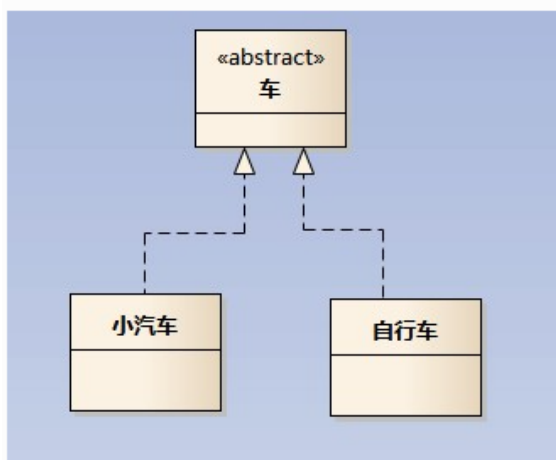
泛化关系用一条带空心箭头的直接表示；如下图表示（A继承自B）；



实现关系(realize)

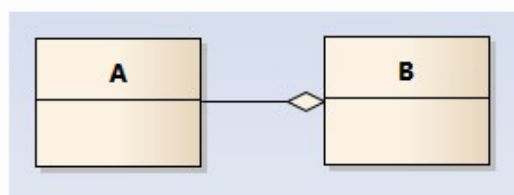
实现关系用一条带空心箭头的虚线表示；

eg：“车”为一个抽象概念，在现实中并无法直接用来定义对象；只有指明具体的子类(汽车还是自行车)，才可以用来定义对象（“车”这个类在C++中用抽象类表示，在JAVA中有接口这个概念，更容易理解）



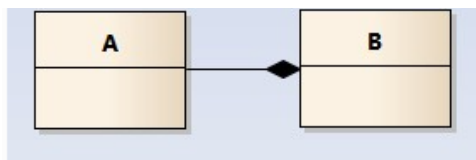
聚合关系(aggregation)

聚合关系用一条带空心菱形箭头的直线表示，如下图表示A聚合到B上，或者说B由A组成；



组合关系(composition)

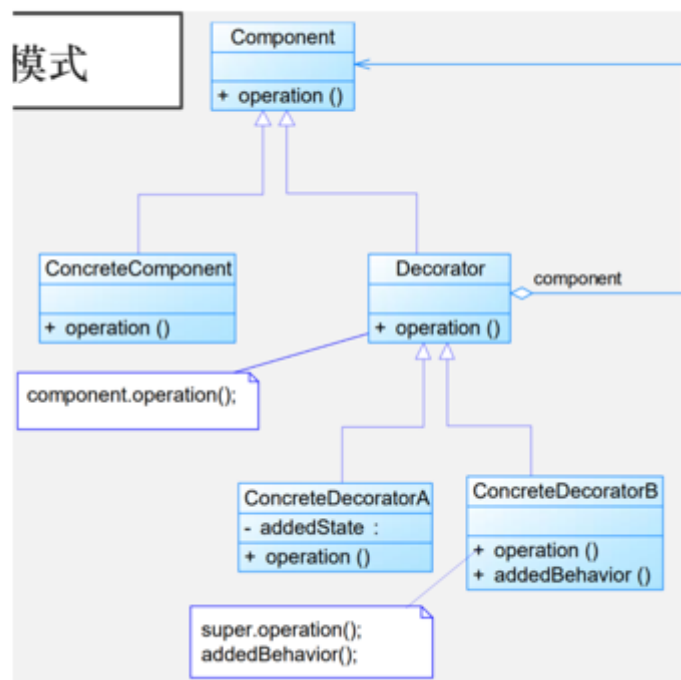
组合关系用一条带实心菱形箭头直线表示，如下图表示A组成B，或者B由A组成；



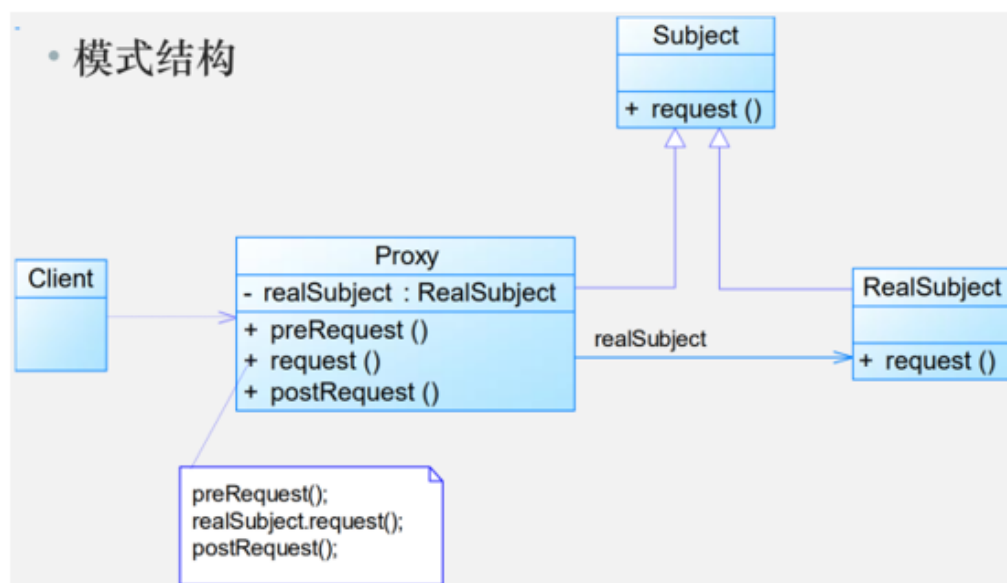
与聚合关系一样，组合关系同样表示整体由部分构成的语义；比如公司由多个部门组成；

但组合关系是一种强依赖的特殊聚合关系，如果整体不存在了，则部分也不存在了；例如，公司不存在了，部门也将不存在了；

15. 装饰者模式



代理模式



1. 防御式编程

1. 防御式编程：可以预见到（或至少预先推测到）问题所在，断定代码中每个阶段可能出现的错误，并作出相应的防范措施，来防止类似意外的发生。
2. 断言：在开发期间使用的、让程序在运行时进行自检的代码，是对开发人员的警告，通常是一个子程序或宏。断言不可以有副作用。

3. 异常：是将代码中的错误或异常事件传递给调用方代码的一种特殊手段，谨慎使用可以降低复杂度。
 4. 错误处理：根据软件类别平衡正确性和健壮性（哪个优先级更高）
 5. 隔离程序：隔离程序是以防御式编程为目的而进行隔离的一种方法，将某些接口选定为“安全”区域的边界，对穿越安全区域边界的数据进行合法性检验（集中工作在特定的模块中降低成本）
 6. 辅助调试代码：辅助进行代码调试的代码，帮助快速检查错误，应该尽早地引入辅助调试代码。
 7. 攻击式编程：主动暴露出可能出现的错误，在开发阶段将其暴露显现出来，而在产品代码运行时让他能够自我恢复。
2. 表驱动法：一种编程模式
 1. 目的：表驱动法适用于复杂逻辑，将复杂逻辑从代码中独立出来方便单独维护
 2. 原理：从表里面查找信息而不使用逻辑语句(if和else)
 3. 具体实现
 1. 直接访问表：直接通过索引值可以从表中找到对应的条目
 2. 索引访问表：首先从索引表中找到数据表的地址，然后再从数据表中找到对应的条目(节省空间、管理廉价、容易维护)
 3. 阶梯访问表：根据每项命中的阶梯层次来确定其归属
 3. 装饰者的两个缺点：很多小类、难以排查错误
 4. 增加新的鸭叫统计次数(不允许修改原本代码)：装饰器模式

4. 选择模式

1. 一个温度系统有3个恒温器，一个恒温器可以调整和显示温度，2个恒温器只能手动调节，1个恒温器可以手动、计时器调节：中介者、策略
 1. 模板方法：先调整、再显示(用户直接复用模板方法，而本例中复用的是实现)
2. 一个队伍有1个管理者和25个队员，每一个队员可以有一个位置，有的特别强的队员可以有多个位置：策略、原型
 1. 策略独立于Player，所以Player是一样的
 2. 模板方法模式是解决步骤的问题
3. 积分会员制，积分比较高则为高级会员，积分比较低则为低级会员

5. 选择模式

What is a Pattern? Match each pattern with its description:

A: Adapter B: State C: Iterator D: Façade E: Strategy F: Observer G: Factory Method H: Decorator I: Command J: Template Method K: Composite L: Singleton

- 1): Wraps an object and provides a different interface to it.
- 2): Subclasses decide how to implement steps in an algorithm.
- 3): Subclasses decide which concrete classes to create.
- 4): Ensures one and only one object is created.
- 5): Encapsulates interchangeable behaviors and use delegation to decide which one to use.
- 6): Clients treat collections of objects and individual objects uniformly.
- 7): Encapsulates state-based behaviors and use delegation to switch between behaviors.
- 8): Provides a way to traverse a collection of objects without exposing its implementation.
- 9): Simplifies the interface of a set of classes.
- 10): Wraps an object to provide new behavior.
- 11): Allows objects to be notified when state changes.
- 12): Encapsulate a request as an object.

Please answer this question by this style: A(13); B(12).....

1. A 适配器模式
2. J 模板方法模式
3. G 工厂方法模式
4. L 单例模式
5. E 策略模式
6. K 组合
7. B 状态
8. C 迭代器
9. D 外观
10. H 装饰者(增加额外行为, 但是接口不变)
11. F 观察者(是一对多的依赖关系, 而不是信息传送, 是信息同步机制)
12. I 命令